The Sampler Was Not the Policy: Finding and Removing a Silent Training–Inference Mismatch in a From-Scratch GRPO Stack

Sankur Kundu
Independent researcher, Bhubaneswar, India
sankur.kundu.tw@gmail.com

Abstract

In reinforcement-learning post-training of language models, one engine samples answers and another computes the gradient, and the policy-gradient estimate is only correct if both hold the same policy. We build the whole loop from scratch, a C++/CUDA inference engine that returns the log-probability of every sampled token and a PyTorch trainer in the engine's own weight layout, so that every source of disagreement between them can be observed. At the released weights of Qwen2.5-0.5B-Instruct the two agree to within 0.041 nats on 16,118 sampled tokens. After one or two GRPO updates the gap reaches 5.2 nats and 0.65% of tokens fall outside a 1 ± 0.2 importance ratio: the engine stores weights in fp16, half an fp16 step around a typical weight (7.6×10^{-6} at 0.02) exceeds an AdamW update at learning rate 10^{-6}, so the engine rounds most of each update away and samples from a policy the trainer never had. Computing the trainer's forward pass with the weights rounded exactly as the engine stores them, while the optimizer keeps float32 master weights, brings the gap back to 0.043 with no token outside the ratio bound. Taking the gap apart over 30 updates, rounding at weight sync accounts for nearly all of it: 9–22$\times$ the effect of fp16 arithmetic and 15–39$\times$ that of the engine's own kernels. An fp16 engine sees 5–13% of weights change in an update; a bf16 engine 0.7–1.7%, and its gap grows to 11 nats with 15% of tokens outside the ratio bound. With the fix, 100 GRPO steps on two T4 GPUs raise GSM8K test accuracy from 48.1% to 52.5%. We also report the systems work that makes such a loop practical on small hardware: in-place weight sync 369$\times$ faster than a reload, partial rollouts with one-step-ahead training that make steps 1.9$\times$ faster, and a sort-free sampler that makes rollouts 27$\times$ faster. Code and raw logs are released.[1]

[1] https://github.com/Shakhtar-Sankur/ratchet

1 Introduction

Policy-gradient post-training of language models, as in PPO (Schulman et al., 2017) and GRPO (Shao et al., 2024), alternates between sampling answers and learning from them. At any scale the two are done by different software: a serving engine optimised for generation (Kwon et al., 2023) and a training framework optimised for gradients (Sheng et al., 2024; Hu et al., 2024). Each computes the probability of the same token in its own way, and the objective's importance ratio is assumed to be one when their weights are equal. When it is not, the gradient is biased and nothing announces it.

Recent work measures this gap at the level of logits and corrects for it, for example by switching the training precision from bf16 to fp16 (Qi et al., 2025). We take a complementary, bottom-up approach: we control both sides completely, so each source of disagreement can be switched on alone and measured on the same sampled tokens. This report makes three contributions.

1. **A from-scratch, observable RL loop** (§2): an inference engine and a trainer that share a weight layout, agree to within 10^{-6} on the CPU, and log the gap on every step of every run.

2. **A silent failure, its cause and its fix** (§3, §6): weight sync into an fp16 engine rounds away most of each update, and this rounding, not arithmetic precision or kernels, is where the gap comes from; computing the trainer with the engine's rounded weights, as mixed-precision training does (Micikevicius et al., 2017), removes it.

3. **Systems measurements on two T4 GPUs** (§4, §5): weight sync, the long tail of answer lengths, sampling cost, and end-to-end accuracy, including the results that are within noise.

2 System

Engine. Rollouts are generated by relay, a C++/CUDA inference engine with paged KV cache and continuous batching, through a C interface. Every sampled token comes with log softmax of the logits divided by the temperature, over the whole vocabulary: the behaviour policy's log-probability. A rollout stopped after any number of tokens and resumed gives the same tokens and log-probabilities as one that ran through, which partial rollouts rely on.

Trainer. The policy is a Llama/Qwen2 decoder in PyTorch whose parameters have the engine's names and fused layout, so a weight sync is a copy with no renaming or reshaping. On four model shapes (multi-head, grouped-query with tied bf16 weights, Qwen2 biases, Llama 3 RoPE scaling) its logits match the `transformers` reference exactly or within 2.1×10^{-7}, and its log-probabilities match the engine's rollouts within 9.5×10^{-7} on the CPU. A whole run is reproducible bit for bit.

GRPO. Each prompt is sampled eight times; rewards are 1 when the final number of the answer equals the reference (Cobbe et al., 2021); advantages are normalised within each group; the loss is PPO's clipped objective averaged over response tokens, optimised with AdamW. Groups whose answers all received the same reward are skipped.

3 The mismatch

At the released weights, the two agree. On a T4 with Qwen2.5-0.5B-Instruct (Qwen Team, 2024), the engine runs in fp16 and the trainer in float32. Over 16,118 sampled tokens the absolute difference between their log-probabilities is at most 0.041 (mean 6.8×10^{-4}, 99th percentile 0.0085), and no token's importance ratio lies outside 1 ± 0.2. The released weights are bf16 values that the engine's fp16 storage represents, so at this point both sides hold the same weights.

After an update, they do not. After one or two GRPO steps, the gap on fresh rollouts reached 4.5 to 6.4 nats on some tokens. The cause is in the weight sync, not the arithmetic. An AdamW step at learning rate 10^{-6} moves a weight by roughly 10^{-6}; for a weight of 0.02, adjacent fp16 values are 1.5×10^{-5} apart, so any change smaller than half of that, 7.6×10^{-6}, rounds back to the old value when the engine converts the trainer's float32 weights. The engine therefore keeps the update only on small weights and samples from a policy that the trainer never had: the trainer computes ratios and gradients for one policy, while the samples come from another.

trainer weights	max	mean	p99	ratio off
float32	5.21	0.0117	0.155	0.65%
fp16-rounded	**0.043**	**0.00067**	**0.0078**	**0**

Table 1: Absolute log-probability gap between engine and trainer on the same 64 fresh rollouts (19,313 tokens) after two GRPO steps. Ratio off: share of tokens whose importance ratio leaves 1 ± 0.2.

The fix. The trainer computes its forward pass with each matrix rounded to fp16 exactly as the engine stores it (round to nearest even), and the optimizer updates float32 master weights through a straight-through gradient, which is the standard mixed-precision recipe (Micikevicius et al., 2017) applied across the trainer–engine boundary. Small updates now accumulate in the master weights until they cross an fp16 step, and both sides always compute with the same values.

Result. Table 1. With fp16-rounded weights the gap after training is what it was at the released weights. In every later run, colocated and split, the per-step maximum stayed between 0.02 and 0.06.

4 Systems

Weight sync. The simple sync copies every parameter to the host, converts it to fp16 on the CPU and rebuilds the engine. Instead, the trainer hands the engine each parameter's device pointer; the engine converts float32 to fp16 on its own GPU, in place, producing the same bits as the host path, and drops its prefix cache, whose entries were computed with the old weights. Updates are refused while a rollout is running, and a later full reload is refused rather than silently reverting the policy. On one T4 with 494M parameters (1.98 GB in float32), the sync takes 0.017 s instead of 6.26 s (369×); across two GPUs, with a peer copy, 0.24 s. Rollouts after either sync are identical.

The long tail. A step waits for its longest answer. Partial rollouts (Kimi Team, 2025) keep more groups generating than a step trains on, end the step when enough have finished, and resume the rest under the new weights from the exact token where they stopped. One-step-ahead training

mode	before	after	time
colocated	48.1%	**52.5%**	50.6 min
split, ahead + partial	47.8%	50.2%	44.7 min

Table 2: Greedy accuracy on all 1,319 GSM8K test problems before and after 100 GRPO steps (8 problems $\times$ 8 answers per step, learning rate 10^{-6}), Qwen2.5-0.5B-Instruct, two T4s. Colocated: both GPUs generate, then train with data parallelism. Split: one generates while the other trains. One run per mode.

generates batch $k{+}1$ while batch k trains, as asynchronous systems do (Fu et al., 2025). Both train on tokens sampled by an older policy, so the importance ratio always uses the engine's sampling-time log-probabilities. On two T4s, one GPU generating while the other trains, a synchronous step takes 49.1 s and a step with both techniques 25.6 s (1.9$\times$), about the training time alone.

Sampling. The first GPU runs generated 34 tokens per second: the engine sorted all 151,936 vocabulary entries on the CPU for every token, which took longer than the forward pass. Sampling at temperature 1 with no top-k or top-p, as RL does, needs no sort; walking the inverse cumulative distribution in token order, with the batch's rows sampled in parallel, takes 1.3 ms per token instead of 29.8 ms, and rollouts run at 931 tokens per second (27$\times$).

5 Results

Table 2. Training reward rose from 0.44 to 0.58 (colocated, first against last ten steps) and from 0.38 to 0.56 (split); answers cut off at 512 tokens fell from 6% to 2%. The same weights scored 48.1% and 47.8% under two batchings of the test set, because fp16 results depend slightly on which sequences share a batch, so differences of a few tenths of a point are noise. With one run per mode, the difference between the two gains is not established.

6 Decomposing the gap by source

Because both sides are ours, each source of disagreement can be switched on alone and measured on the same sampled tokens. *Weight rounding* compares the trainer with its float32 master weights against the same trainer with those weights rounded to fp16, as the engine stores them, or to bf16. *Arithmetic precision* compares fp16 autocast with float32 at identical weights. *Engine kernels* is what remains between the engine and the closest trainer variant. Each is measured on 64 fresh rollouts (8 problems $\times$ 8 answers, temperature 1) at the released weights and after 1, 3, 10 and 30 GRPO updates of a run with the fix, on two T4s. We also count, after every update, the share of weight-matrix entries whose rounded value changes: the part of the update the engine can see.

Rounding at sync is the gap. At the released weights every source is small, and rounding is zero for bf16 because the released weights are bf16 values. After a single update, rounding the same weights to fp16 changes per-token log-probabilities by 0.016 on average and by up to 1.9 nats, with 1.2% of tokens outside the ratio bound (Table 3). Arithmetic precision and the engine's kernels stay where they were at every checkpoint, at most 0.14 and 0.07 nats with no token outside the bound; on average, rounding is 9–22$\times$ larger than arithmetic and 15–39$\times$ larger than the kernels. The engine is always closest to the trainer variant that uses its own rounded weights, which is the fix of §3: with it, the engine and trainer agree within 0.031–0.066 nats at every checkpoint. A second run without the fix reached 1.8 to 4.6 nats between engine and trainer by updates 1–10, as in §3.

How much of an update an engine sees. Once rounded to fp16, only 13.4% of weight-matrix entries change value in the first update, falling to 5.0% by update 30 as the optimizer's steps shrink; with the fix, the rest of each update accumulates in the master weights until it crosses an fp16 step. In bf16, which has three fewer mantissa bits, 1.7% change in the first update and 0.7% by update 30, about eight times fewer. These shares count every entry, including any the update did not move, such as embedding rows of tokens absent from the batch. From the first update on, 97–99.8% of master weights lie off either grid.

bf16 weights would be worse, and grow worse. Had the engine stored bf16 weights, the gap from rounding alone would have been larger at every checkpoint and grown with training, to a mean of 0.20 and a maximum of 11 nats, with 15% of tokens outside the ratio bound by update 30, while the fp16 gap stayed between 0.012 and 0.030. This agrees with the finding that fp16 reduces the training–inference mismatch relative to bf16 (Qi et al., 2025); here the cause can be seen directly, in the weights the engine holds. Why the bf16 gap grows in this run is not established by one run.

	weight rounding, fp16			weight rounding, bf16			arithmetic		engine kernels	
updates	mean	max	off	mean	max	off	mean	max	mean	max
0	0.000	0.0001	0	0	0	0	0.0013	0.081	0.0007	0.040
1	0.016	1.93	1.2%	0.022	2.16	2.2%	0.0012	0.073	0.0007	0.033
3	0.025	2.30	2.8%	0.041	3.65	5.5%	0.0012	0.136	0.0007	0.056
10	0.030	3.47	3.4%	0.104	9.70	13.3%	0.0014	0.064	0.0008	0.031
30	0.012	0.78	1.0%	0.196	10.98	15.0%	0.0013	0.069	0.0008	0.066

Table 3: The engine–trainer gap taken apart: absolute difference in per-token log-probability (nats) caused by each source alone, on 64 fresh rollouts per checkpoint of one run. Off: share of tokens whose importance ratio leaves 1 ± 0.2 (none for arithmetic or kernels at any checkpoint).

Where the gap falls. Without the fix, the gap concentrates on the tokens the engine sampled with low probability: a mean of 0.10–0.17 nats for tokens it gave less than 0.1, against 0.001–0.002 for tokens above 0.9. These are the exploratory tokens whose importance ratios the clipped objective acts on.

7 Discussion

The failure in §3 is easy to miss: training still runs, rewards still move, and the only symptom is a gap that most stacks do not measure. Section 6 shows where to look: not at the arithmetic or the kernels, which stay within 0.14 nats, but at the weights each side actually holds after a sync. It is also easy to cause, because storing inference weights in a lower precision than the optimizer's is common in systems that separate the two. Two habits catch it: logging the engine–trainer gap on fresh samples at every step, and computing the trainer's forward pass with the weights the engine will actually hold. The second costs one rounding per matrix per step.

Limitations

All results use one 0.5B model, one task and two T4 GPUs, with one run per training mode. The decomposition in §6 uses 64 rollouts per checkpoint from one run; its bf16 column measures rounding to bf16 in our trainer, not a bf16 engine. Larger models and other learning rates change how much of an update survives rounding.

Use of AI assistants

AI coding and writing assistants were used to help write code and prose. The author designed the study, checked every reported number against the released logs, and takes responsibility for the content.

References

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, et al. 2025. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*.

Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, et al. 2024. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. *arXiv preprint arXiv:2405.11143*.

Kimi Team. 2025. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, et al. 2023. Efficient memory management for large language model serving with PagedAttention. *arXiv preprint arXiv:2309.06180*.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, et al. 2017. Mixed precision training. *arXiv preprint arXiv:1710.03740*.

Penghui Qi, Zichen Liu, Xiangxin Zhou, Tianyu Pang, et al. 2025. Defeating the training-inference mismatch via FP16. *arXiv preprint arXiv:2510.26788*.

Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, et al. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, et al. 2024. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint arXiv:2409.19256*.